Azure Multi-VM DevOps Lab Guide: Jenkins, Open Source Puppet & CI/CD

Workshop Duration: 4 Hours | **Platform:** Ubuntu 22.04 LTS on Microsoft Azure

Lab Topology & Objectives

This lab guide provides step-by-step instructions for establishing an automated CI/CD pipeline between two Azure virtual machines. VM 1 acts as the Jenkins Controller, while VM 2 acts as the Open Source Puppet Server target. Learners will configure passwordless SSH authentication, set up r10k with GitHub repositories, and execute automated deployments via Jenkins pipelines.

Step 1: Azure VM Provisioning & Network Security Groups

1. Create two Ubuntu 22.04 LTS VMs in the same Azure resource group: **jenkins-controller-vm** (VM 1) and **puppet-target-vm** (VM 2).

2. Configure Network Security Groups (NSG) to allow inbound traffic on **Port 22** (SSH) for both VMs, **Port 8080** (Jenkins) for VM 1, and **Port 80** (HTTP) for VM 2.

Step 2: Install Jenkins on Azure VM 1

Connect to VM 1 via SSH and execute the installation commands:

```
sudo apt update && sudo apt install fontconfig openjdk-17-jre -y
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list >
/dev/null
sudo apt update && sudo apt install jenkins -y
sudo systemctl enable --now jenkins
```

Retrieve the initial administrator password using: sudo cat /var/lib/jenkins/secrets/initialAdminPassword and complete setup at http://<VM1-IP>:8080.

Step 3: Configure Dedicated deploy-user on Azure VM 2

Connect to VM 2 via SSH and create a non-root deployment user with passwordless sudo rights:

```
sudo useradd -m -s /bin/bash deploy-user
echo "deploy-user ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/deploy-user
sudo chmod 440 /etc/sudoers.d/deploy-user
sudo mkdir -p /home/deploy-user/.ssh && sudo chmod 700 /home/deploy-user/.ssh
sudo touch /home/deploy-user/.ssh/authorized_keys && sudo chmod 600
/home/deploy-user/.ssh/authorized_keys
sudo chown -R deploy-user:deploy-user /home/deploy-user/.ssh
```

Step 4: Establish Passwordless SSH from VM 1 to VM 2

On VM 1, switch to the jenkins user and generate an SSH key pair:

```
sudo su - jenkins -s /bin/bash
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
```

Copy the generated public key and append it to /home/deploy-user/.ssh/authorized_keys on VM 2.

Step 5: Configure r10k and GitHub SSH on VM 2

Ensure VM 2 has an SSH deploy key linked to your GitHub repository and configure /etc/r10k.yaml:

```
---
:cachedir: '/opt/puppetlabs/server/data/r10k/cache'
:sources:
:my-control-repo:
:remote: 'git@github.com:/puppet-control-repo.git'
:prefix: false
```

Step 6: Create the End-to-End Jenkins Pipeline Job

Create a new Pipeline job in Jenkins and paste the following declarative pipeline script:

```
pipeline {
agent any
stages {
stage('Validate Manifest Syntax') {
steps {
sh 'puppet parser validate manifests/site.pp || true'
}
}
stage('Deploy to Target VM via SSH') {
steps {
sh '''
ssh -o StrictHostKeyChecking=no deploy-user@ "
sudo r10k deploy environment production -p --verbose &&
sudo puppet apply /etc/puppetlabs/code/environments/production/manifests/site.pp
"
'''
}
}
}
}
```